

CS331 – COMPUTER NETWORKS

Intelligent Network Configuration Assistant using MCP and Rule-Based Automation

Project Report

Team ID T015
Project ID 5
Course CS331 – Computer Networks

Team Members

Name	Roll Number
Kushal Kumar	24110178
Kshitij Srivastava	24110174
Pandit Shreyash Malhari	24110334
Vishaw Raj Meena	24110388
Tejas Prasad	24110369

Abstract

Network administration frequently involves repetitive operations such as checking connectivity, modifying firewall rules, and controlling bandwidth. This project presents an Intelligent Network Configuration Assistant that allows administrators to perform such operations through natural-language commands.

The system combines a local Large Language Model (LLM), the Model Context Protocol (MCP), deterministic rule-based policy validation, and standard Linux networking utilities. The LLM interprets the user's request and selects a structured MCP tool, while the MCP server exposes controlled operations for firewall management, connectivity testing, traffic shaping, and bandwidth measurement.

A key design principle of the system is that the LLM is not given unrestricted shell access. Instead, every requested operation passes through structured tool-call validation and a server-side policy engine before execution. Network commands are executed using fixed argument vectors and are followed by verification whenever possible. The system also supports a safe dry-run mode, explicit live execution, Docker-based deployment, and JSON Lines audit logging.

The implementation was validated on a real Wi-Fi network. Connectivity testing achieved zero packet loss with an average RTT of approximately 23.985 ms. A baseline bandwidth of approximately 40 Mbps was measured using `iperf3`; after applying a 5 Mbps traffic-control limit using `tc`, measured throughput reduced to approximately 5 Mbps. These results demonstrate an end-to-end workflow from natural-language input to real network configuration and independent verification.

Contents

1	Introduction	4
2	Problem Statement	4
3	Objectives	5
4	System Overview	5
5	System Architecture	6
6	Technologies Used	7
7	Model Context Protocol Implementation	7
7.1	Why MCP?	7
7.2	MCP Communication	7
8	Implemented MCP Tools	8
8.1	block_ip	8
8.2	unblock_ip	8
8.3	list_firewall_rules	8
8.4	ping_host	9
8.5	limit_bandwidth	9
8.6	test_bandwidth	9
9	Rule-Based Policy Engine	9
10	Security Design	10
10.1	No Arbitrary Shell Access	10
10.2	Argument-Vector Execution	10
10.3	Protected IP Addresses	11
10.4	Interface Allow-List	11
10.5	Bandwidth Bounds	11
11	LLM Output Validation	11
12	Command Runner	12
12.1	Dry-Run Mode	12
12.2	Live Mode	12
13	Audit Logging	12
14	Docker Deployment	13
14.1	Host Networking	13
14.2	Docker Workflow	14
15	Experimental Network Environment	14

16 Experimental Validation	14
16.1 Connectivity Test	14
16.2 Firewall Blocking Test	15
16.3 Bandwidth Limiting Test	15
17 Results and Discussion	16
18 Testing	17
18.1 Policy Unit Tests	17
18.2 Integration Testing	17
19 Failure Handling	17
19.1 Malformed LLM Output	17
19.2 Policy Violation	18
19.3 Ping Failure	18
19.4 iperf3 Failure	18
19.5 Docker Failure	18
20 Non-Functional Requirements	18
20.1 Security	18
20.2 Reliability	18
20.3 Auditability	18
20.4 Safety	19
20.5 Maintainability	19
20.6 Usability	19
20.7 Portability	19
21 Limitations	19
22 Future Enhancements	20
23 Complete Operational Workflow	20
24 Implementation Structure	21
25 Demonstration Procedure	22
26 Conclusion	23
A Appendix: Policy Configuration	25
B Appendix: Example Commands	25

1 Introduction

Modern computer networks require administrators to perform many routine configuration and monitoring tasks. Examples include checking whether a host is reachable, blocking an unwanted source address, removing a firewall rule, limiting traffic, and measuring available bandwidth.

Traditionally, these operations require knowledge of operating-system commands and their correct parameters. This creates an opportunity for an intelligent assistant that can translate natural-language requests into structured network-management operations.

The objective of this project is to develop such an assistant while avoiding the security risks associated with giving an LLM unrestricted access to a shell. The proposed system therefore separates natural-language interpretation from deterministic policy enforcement and command execution.

The project uses the Model Context Protocol (MCP) to expose network-management operations as structured tools. A local Qwen3:8b model running through Ollama interprets user requests. A FastMCP server provides the available network tools. A rule-based policy layer validates requested parameters before the corresponding Linux utilities are executed.

The system focuses particularly on the following non-functional properties:

- Security
- Reliability
- Auditability
- Safety
- Maintainability
- Usability

2 Problem Statement

Network administrators commonly need to perform operations such as:

- blocking or unblocking an IP address,
- checking connectivity to another host,
- applying bandwidth restrictions,
- measuring network throughput, and
- inspecting existing firewall rules.

Performing these operations manually requires familiarity with tools such as `iptables`, `tc`, `ping`, and `iperf3`.

An LLM can provide a natural-language interface to these operations, but directly allowing an LLM to execute arbitrary shell commands introduces security and reliability problems. An incorrectly interpreted command or malformed model output could result in unintended network changes.

Therefore, the problem addressed by this project is:

How can a natural-language network administration assistant use an LLM to select network operations while ensuring that actual network changes remain deterministic, policy-controlled, auditable, and verifiable?

3 Objectives

The main objectives of the project are:

1. Build a command-line network administration assistant.
2. Integrate a local LLM for natural-language interpretation.
3. Implement the Model Context Protocol for structured tool access.
4. Provide controlled tools for common network operations.
5. Introduce a deterministic policy layer for authorization.
6. Prevent unrestricted shell access by the LLM.
7. Validate LLM-generated tool calls before execution.
8. Verify network configuration after changes.
9. Record executed commands in an audit log.
10. Provide dry-run and live execution modes.
11. Containerize the MCP server using Docker.
12. Validate the implementation on a real network.

4 System Overview

The system consists of the following major components:

- **User Interface:** A command-line interface through which the administrator enters natural-language requests.
- **LLM Layer:** Qwen3:8b running locally through Ollama. The LLM interprets the user's request and selects an appropriate tool.
- **MCP Client:** Implemented in `assistant.py`. It discovers the available tools and communicates with the MCP server.
- **MCP Server:** Implemented using FastMCP and deployed inside Docker. It exposes structured network-management tools.
- **Policy Engine:** Validates IP addresses, protected addresses, interfaces, and bandwidth limits.
- **Network Tool Layer:** Executes controlled operations using Linux networking utilities.
- **Audit Logger:** Records command execution and results in JSON Lines format.

5 System Architecture

The high-level architecture is shown below.

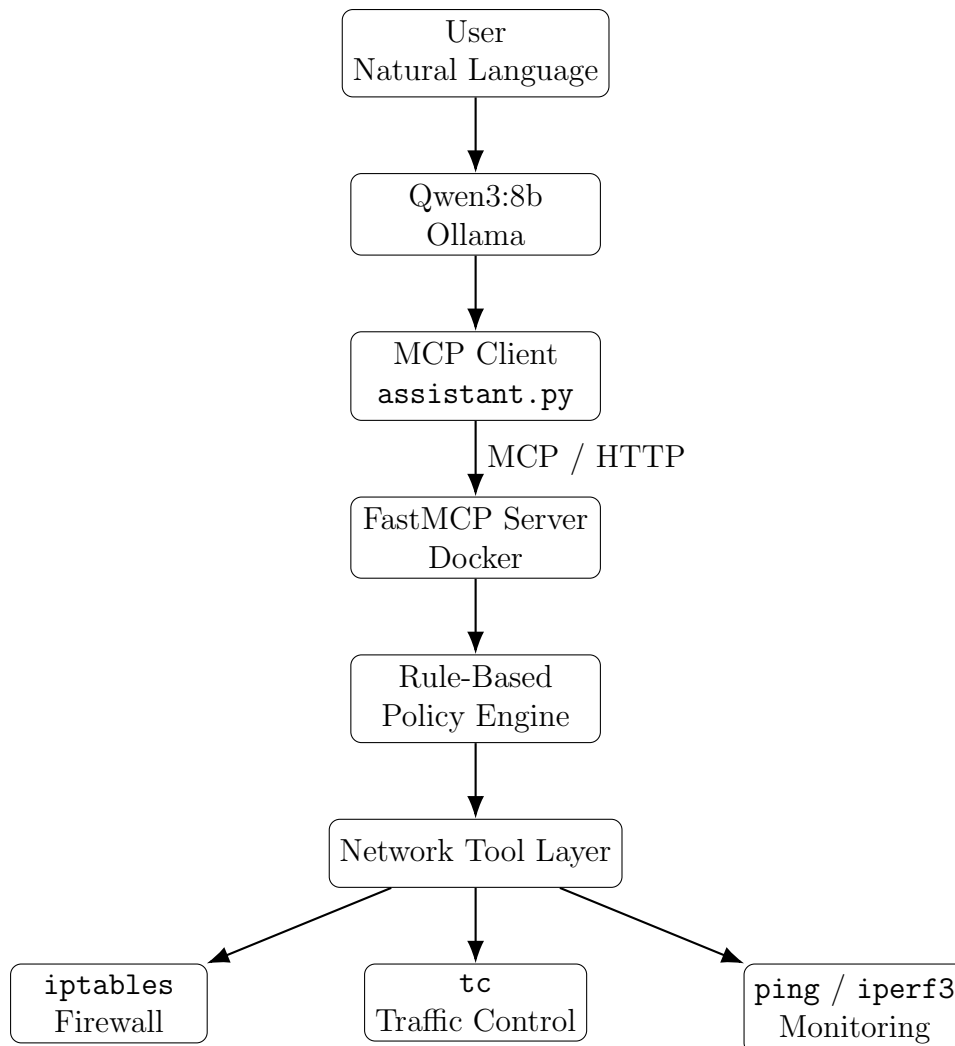


Figure 1: High-level architecture of the Intelligent Network Configuration Assistant

The architecture deliberately separates interpretation from execution.

The LLM is responsible for understanding the user's intent and selecting a structured tool. It does not directly execute arbitrary shell commands.

The policy engine provides a deterministic authorization boundary between the LLM and the operating system.

6 Technologies Used

Table 1: Technologies used in the project

Technology	Purpose
Python 3	Application and automation logic
Qwen3:8b	Local LLM for natural-language interpretation
Ollama	Local LLM runtime
FastMCP	MCP client and server implementation
MCP	Structured communication between assistant and network tools
iptables	Firewall rule management
tc	Linux traffic-control and bandwidth limiting
ping	Connectivity testing
iperf3	Network throughput measurement
YAML	Policy configuration
Docker	Containerized MCP deployment
Docker Compose	Container lifecycle management
pytest	Automated testing

7 Model Context Protocol Implementation

7.1 Why MCP?

The Model Context Protocol provides a structured mechanism through which an AI application can discover and invoke external tools.

In this project, MCP is used to create a clear boundary between:

- the natural-language reasoning performed by the LLM, and
- the deterministic network operations performed by the server.

The assistant dynamically discovers the tools exposed by the FastMCP server. This makes the available network operations explicit instead of requiring the LLM to generate arbitrary shell commands.

7.2 MCP Communication

The current deployment uses HTTP-based MCP communication.

The assistant connects to:

```
http://localhost:8000/mcp
```

The FastMCP server runs inside the Docker container and listens on port 8000.

The communication flow is:

1. User enters a natural-language request.

2. Qwen3:8b interprets the request.
3. The LLM selects an MCP tool and generates its arguments.
4. The assistant validates the generated tool call.
5. The MCP client sends the structured call to the server.
6. The server performs policy validation.
7. The network operation is executed if permitted.
8. The result is returned through MCP.
9. The LLM converts the result into a user-readable response.

8 Implemented MCP Tools

The system currently exposes six MCP tools.

8.1 `block_ip`

This tool blocks inbound traffic from an authorized source IP.

Conceptually, it performs:

```
sudo -n iptables -I INPUT -s <IP> -j DROP
```

The rule is checked after application to verify that it exists.

Example natural-language request:

```
block 10.7.33.21
```

8.2 `unblock_ip`

This tool removes the assistant's inbound DROP rule.

Conceptually:

```
sudo -n iptables -D INPUT -s <IP> -j DROP
```

The implementation also checks that the rule is no longer present.

8.3 `list_firewall_rules`

This tool lists the current INPUT-chain firewall rules.

```
sudo -n iptables -S INPUT
```

8.4 ping_host

This tool checks whether an authorized host is reachable.

It uses the Linux `ping` utility and reports information such as:

- packet transmission,
- packets received,
- packet loss,
- minimum RTT,
- average RTT, and
- maximum RTT.

8.5 limit_bandwidth

This tool applies an outbound bandwidth limit using Linux traffic control (`tc`).

For the demonstrated network, the command:

```
limit 10.7.5.27 to 5 Mbps on wlo1
```

limits traffic originating from the local IP address `10.7.5.27` on interface `wlo1`.

The implementation creates an HTB qdisc, an HTB class, and a traffic-control filter matching the source IP.

An important aspect of the implementation is that the limit is **outbound and source-IP based**. Therefore, for traffic sent from `10.7.5.27` to `10.7.33.21`, the local source address is the address that is limited.

8.6 test_bandwidth

This tool uses `iperf3` to measure throughput to a peer.

The peer must first run:

```
iperf3 -s
```

The assistant then performs an `iperf3` client test against the peer.

9 Rule-Based Policy Engine

The policy engine is one of the most important safety components.

The LLM does not have authority to decide whether an operation is permitted. Instead, the server performs deterministic validation.

The current policy is:

```
allowed_networks:
- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

protected_ips:
```

```
- 127.0.0.1
- 192.168.1.1

allowed_interfaces:
- eth0
- ens33
- enp0s3
- wlo1

min_bandwidth_mbps: 1
max_bandwidth_mbps: 100
```

The policy engine validates:

- IPv4 addresses,
- authorized network membership,
- protected IP addresses,
- interface names, and
- bandwidth limits.

For example, an attempt to operate on a protected address is rejected even if the LLM selects a valid MCP tool.

This gives the system a deterministic safety boundary.

10 Security Design

Security is a major design goal of the project.

10.1 No Arbitrary Shell Access

The LLM does not generate a shell command and execute it directly.

Instead, the execution path is:

**Natural Language → LLM Tool Selection → Tool Validation → MCP → Policy
→ Fixed Command → Execution**

10.2 Argument-Vector Execution

Network commands are passed as argument lists rather than shell-interpolated strings.

For example:

```
["sudo", "-n", "iptables", "-I", "INPUT",  
"-s", ip, "-j", "DROP"]
```

This avoids passing user-controlled data through a shell interpreter.

10.3 Protected IP Addresses

The policy contains protected addresses such as:

```
127.0.0.1
192.168.1.1
```

These addresses cannot be manipulated through the assistant's authorized operations.

10.4 Interface Allow-List

Only interfaces specified by the policy are accepted.

The demonstrated interface was:

```
wlo1
```

10.5 Bandwidth Bounds

The policy limits bandwidth values to:

$$1 \leq B \leq 100 \text{ Mbps}$$

This prevents obviously unreasonable values from being passed to the traffic-control layer.

11 LLM Output Validation

Because LLM output is probabilistic, the assistant validates every generated tool call before sending it to the MCP server.

The validation layer checks:

1. whether the output can be parsed,
2. whether the selected tool exists,
3. whether all required parameters are present,
4. whether unexpected parameters are present,
5. whether parameters have the expected basic types, and
6. whether arguments can be safely converted into the expected structured representation.

Examples of invalid model outputs include:

```
Unknown tool: delete_everything
```

or:

```
Missing required parameter: ip
```

or:

```
Unexpected parameter: shell_command
```

Such calls are rejected before an MCP network operation is performed.

The design principle is:

LLM decision $\neq$ authorized network action

The LLM proposes the operation, while deterministic validation and policy rules control execution.

12 Command Runner

The command runner provides a centralized mechanism for executing network commands.

It has two execution modes.

12.1 Dry-Run Mode

In dry-run mode, the command is constructed and recorded but is not executed.

The default environment is:

```
NETWORK_ASSISTANT_EXECUTE=0
```

This makes dry-run the safe default.

12.2 Live Mode

Actual execution is enabled explicitly using:

```
NETWORK_ASSISTANT_EXECUTE=1
```

For Docker Compose, the mode can be selected without editing the compose file:

```
NETWORK_ASSISTANT_EXECUTE=0 docker compose up
```

or:

```
NETWORK_ASSISTANT_EXECUTE=1 docker compose up
```

Live mode is intended for the controlled laboratory environment.

13 Audit Logging

The system records command execution in:

```
logs/audit.jsonl
```

Each record contains information such as:

- timestamp,
- command,
- success or failure,

- return code,
- standard output,
- standard error, and
- dry-run status when applicable.

The log directory is created automatically by the command runner.

This provides traceability for network operations and supports the auditability requirement of the system.

Runtime logs are not committed to the project repository because they are environment-specific.

14 Docker Deployment

The MCP server is containerized using Docker.

The Docker image is based on Ubuntu 24.04 and installs:

- Python 3,
- FastMCP dependencies,
- PyYAML,
- iptables,
- iproute2,
- iperf3,
- iputils-ping, and
- sudo.

The Docker container runs the FastMCP server.

14.1 Host Networking

The controlled laboratory deployment uses:

```
network_mode: host
```

Host networking is used because the project needs to demonstrate actual traffic-control and firewall operations on the host network interface `wlo1`.

The container is also granted:

```
cap_add:  
- NET_ADMIN  
- NET_RAW
```

These capabilities are required for the demonstrated network operations.

This deployment should therefore be treated as a controlled laboratory configuration rather than a production-ready security boundary.

14.2 Docker Workflow

The server is built using:

```
docker compose build
```

and started using:

```
docker compose up
```

The MCP endpoint is:

```
http://localhost:8000/mcp
```

15 Experimental Network Environment

The system was tested using two laptops connected to the same Wi-Fi network.

Table 2: Experimental network configuration

Device	IP Address	Interface
Assistant laptop	10.7.5.27/18	wlo1
Peer laptop	10.7.33.21/18	Windows

The Linux routing table showed:

```
10.7.0.0/18 dev wlo1 src 10.7.5.27
```

The neighbor table also contained the peer:

```
10.7.33.21 dev wlo1
```

This confirmed Layer-2 and Layer-3 connectivity between the two systems.

16 Experimental Validation

The project was tested in live mode on the experimental network.

Three major experiments were performed:

1. Connectivity verification using `ping`.
2. Firewall blocking and unblocking using `iptables`.
3. Bandwidth limiting and measurement using `tc` and `iperf3`.

16.1 Connectivity Test

The assistant was asked:

```
what is the ping of 10.7.33.21
```

The MCP server executed the connectivity test.
The observed result was:

Table 3: Ping verification result

Metric	Result
Packets transmitted	3
Packets received	3
Packet loss	0%
Minimum RTT	23.061 ms
Average RTT	23.985 ms
Maximum RTT	24.485 ms

The result confirms successful connectivity between the two devices.

16.2 Firewall Blocking Test

The assistant was given:

```
block 10.7.33.21
```

The LLM selected:

```
block_ip
```

The MCP server applied the corresponding `iptables` DROP rule.

The assistant then returned a successful result indicating that the IP had been blocked.

The rule was subsequently removed using:

```
unblock 10.7.33.21
```

The successful execution demonstrated the complete path:

Natural Language → Qwen3 → MCP → Policy → `iptables`

16.3 Bandwidth Limiting Test

The baseline throughput between the two laptops was first measured using `iperf3`.

The baseline result was approximately:

Table 4: Baseline bandwidth

Metric	Throughput
Sender	40.9 Mbit/sec
Receiver	39.5 Mbit/sec

The assistant was then asked to apply a bandwidth limit:

```
limit 10.7.5.27 to 5 Mbps on wlo1
```

The tool configured an HTB class with:

```
rate 5Mbit
ceil 5Mbit
```

Independent verification using:

```
sudo tc class show dev wlo1
```

produced:

```
class htb 1:10 root prio 0 rate 5Mbit ceil 5Mbit
```

A second `iperf3` measurement was then performed.

The measured throughput was:

Table 5: Bandwidth after applying the 5 Mbps limit

Metric	Throughput
Sender	5.35 Mbit/sec
Receiver	4.66 Mbit/sec

The experiment therefore demonstrated a reduction from approximately 40 Mbps to approximately 5 Mbps.

17 Results and Discussion

The experimental results demonstrate that the system can perform actual network operations from natural-language commands while maintaining a structured authorization path.

Table 6: Summary of experimental results

Test	Result
MCP tool discovery	Six network tools discovered successfully
Ping test	3/3 packets received, 0% loss
Average RTT	23.985 ms
Firewall block	DROP rule applied successfully
Firewall unblock	DROP rule removed successfully
Baseline bandwidth	Approximately 40 Mbps
Configured limit	5 Mbps
Post-limit bandwidth	Approximately 5 Mbps
Automated policy tests	7 tests passed

The bandwidth experiment is particularly significant because it provides independent behavioral verification. The system did not merely report that the `tc` command had executed successfully; the resulting throughput also changed to approximately the configured limit.

18 Testing

Testing was performed at multiple levels.

18.1 Policy Unit Tests

The policy implementation was tested using `pytest`.

The final test run produced:

7 passed

The tests cover important policy conditions such as:

- valid network addresses,
- unauthorized addresses,
- protected IP addresses,
- valid interfaces,
- invalid interfaces,
- valid bandwidth values, and
- invalid bandwidth values.

18.2 Integration Testing

The complete system was also tested manually using:

1. natural-language user input,
2. LLM tool selection,
3. MCP tool discovery,
4. Dockerized MCP execution,
5. policy validation,
6. actual network configuration, and
7. independent network verification.

19 Failure Handling

The system is designed to return explicit failure information rather than silently assuming that an operation succeeded.

19.1 Malformed LLM Output

Malformed or invalid tool calls are rejected before execution.

19.2 Policy Violation

If an IP, interface, or bandwidth value violates the configured policy, the MCP tool returns a rejection response.

19.3 Ping Failure

Ping failure may occur if:

- the peer is offline,
- the IP address is incorrect,
- a firewall blocks ICMP, or
- the network connection is unavailable.

19.4 iperf3 Failure

An `iperf3` test requires an active server on the peer.

The peer must run:

```
iperf3 -s
```

If the server is not running or the connection is blocked, the test may fail or time out. This is treated as an operational failure and is returned to the assistant.

19.5 Docker Failure

Docker status and logs can be inspected using:

```
docker compose ps  
docker compose logs
```

20 Non-Functional Requirements

The project places significant emphasis on non-functional requirements.

20.1 Security

The system avoids unrestricted shell execution by the LLM. Network actions are exposed only through predefined MCP tools and are subject to deterministic policy validation.

20.2 Reliability

Operations provide explicit success or failure results. Configuration changes are verified whenever practical.

20.3 Auditability

Network command execution is recorded in a timestamped JSON Lines audit log.

20.4 Safety

Dry-run mode is enabled by default. Live execution requires explicit configuration.

20.5 Maintainability

The implementation is divided into separate modules for:

- network operations,
- policy validation,
- command execution,
- MCP server functionality, and
- LLM interaction.

This modular structure allows individual components to be modified or tested without redesigning the complete system.

20.6 Usability

The assistant accepts natural-language commands instead of requiring the administrator to remember complete Linux networking command syntax.

20.7 Portability

The MCP server is packaged using Docker, while the underlying network operations use standard Linux utilities.

21 Limitations

The current implementation is primarily intended for an academic and controlled laboratory environment.

1. Live network operations are Linux-specific.
2. Docker host networking reduces network-namespace isolation.
3. The container requires network administration capabilities for live operations.
4. The current traffic-control configuration is simplified.
5. `iperf3` requires a server process on the peer.
6. The policy file is designed for the demonstrated lab environment.
7. The current audit log is local rather than centrally collected.
8. The LLM can still misunderstand ambiguous natural-language intent. Deterministic validation reduces unsafe execution but cannot guarantee perfect semantic interpretation.

22 Future Enhancements

Possible improvements include:

1. Adding a dedicated `remove_bandwidth_limit` MCP tool.
2. Adding comprehensive integration tests for LLM tool-call validation.
3. Introducing centralized audit logging.
4. Adding role-based authorization.
5. Supporting richer network-context discovery.
6. Adding persistent configuration state.
7. Providing rollback mechanisms for network changes.
8. Supporting more sophisticated traffic-control policies.
9. Adding monitoring and alerting.
10. Providing a web-based administration interface.
11. Building a multi-host containerized network test environment.

23 Complete Operational Workflow

The complete operational workflow can be summarized as follows.

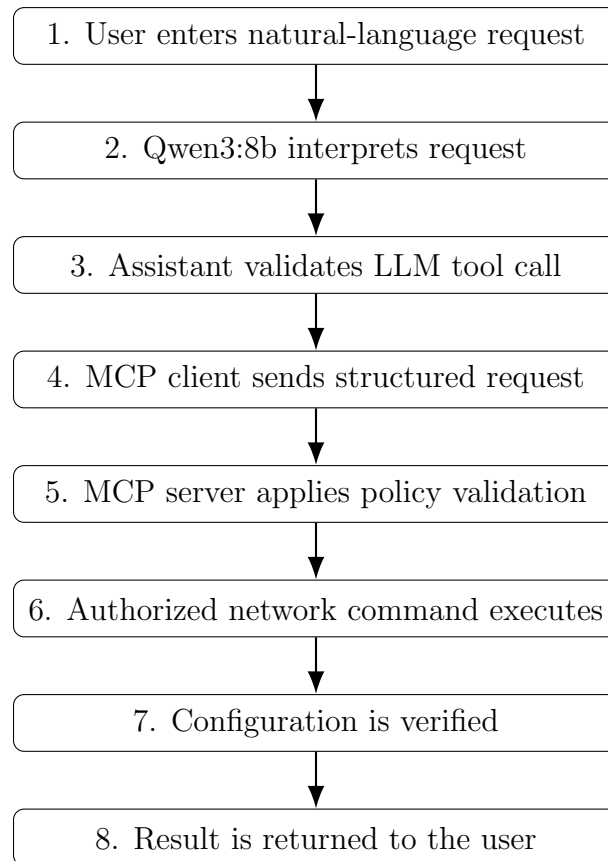


Figure 2: End-to-end operational workflow

This layered design ensures that the LLM remains an interpreter rather than an unrestricted system administrator.

24 Implementation Structure

The project is organized into separate components.

```
network-mcp-assistant/  
|  
+-- assistant.py  
+-- cli.py  
+-- requirements.txt  
+-- Dockerfile  
+-- docker-compose.yml  
|  
+-- core/  
|   +-- __init__.py  
|   +-- network_tools.py  
|   +-- policy.py  
|   +-- runner.py  
|  
+-- server/  
|   +-- mcp_server.py  
|
```

```
+-- policies/
|   +-- network_policy.yaml
|
+-- tests/
|   +-- test_policy.py
|
+-- docs/
|   +-- demo-plan.md
|
+-- logs/
    +-- audit.jsonl
```

The runtime `logs/` directory is generated automatically and is not required to be committed to the source repository.

25 Demonstration Procedure

A complete demonstration can be performed using the following sequence.

Step 1: Start the MCP Server

```
docker compose up
```

Step 2: Start the Assistant

```
source .venv/bin/activate
python assistant.py
```

Step 3: Test Connectivity

```
what is the ping of 10.7.33.21
```

Step 4: Block the Peer

```
block 10.7.33.21
```

Step 5: Verify the Block

Run the ping request again and observe the connectivity failure.

Step 6: Unblock the Peer

```
unblock 10.7.33.21
```

Step 7: Start iperf3 on the Peer

```
iperf3 -s
```

Step 8: Measure Baseline Bandwidth

```
what is my bandwidth with 10.7.33.21
```

Step 9: Apply the Bandwidth Limit

```
limit 10.7.5.27 to 5 Mbps on wlo1
```

Step 10: Verify Traffic Control

```
sudo tc class show dev wlo1
```

Step 11: Measure Bandwidth Again

```
what is my bandwidth with 10.7.33.21
```

The measured throughput should be close to the configured 5 Mbps limit, subject to normal network variation.

26 Conclusion

This project presents an Intelligent Network Configuration Assistant that combines natural-language interaction, a local LLM, MCP, deterministic policy-based validation, and standard Linux networking tools.

The central design decision was to avoid giving the LLM unrestricted shell access. Instead, the LLM selects structured MCP tools, while the server-side policy engine determines whether the requested operation is authorized.

The system provides six network-management tools for firewall operations, connectivity testing, bandwidth limiting, and throughput measurement. It also supports dry-run execution, explicit live mode, audit logging, and Docker-based deployment.

The real-network experiments demonstrated that the system can perform actual network configuration rather than only simulating commands. Connectivity was successfully verified with zero packet loss and an average RTT of 23.985 ms. A baseline throughput of approximately 40 Mbps was measured and was subsequently reduced to approximately 5 Mbps after applying a 5 Mbps `tc` limit.

The project therefore demonstrates a complete pipeline:

Natural Language → LLM → MCP → Policy → Network Tool → Verification

This architecture provides a practical foundation for intelligent network automation while preserving important security, reliability, auditability, and safety properties.

References

1. Model Context Protocol documentation and specification.
2. FastMCP documentation for Python-based MCP server and client implementation.
3. Ollama documentation for local execution of large language models.
4. Linux `iptables` documentation for firewall configuration.
5. Linux `tc` documentation for traffic control and bandwidth shaping.
6. `iperf3` documentation for network throughput measurement.
7. Docker documentation for containerized application deployment.
8. Python documentation for subprocess execution and application development.

A Appendix: Policy Configuration

The policy used during the demonstrated experiment was:

```
allowed_networks:
- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

protected_ips:
- 127.0.0.1
- 192.168.1.1

allowed_interfaces:
- eth0
- ens33
- enp0s3
- wlo1

min_bandwidth_mbps: 1
max_bandwidth_mbps: 100
```

B Appendix: Example Commands

Docker

```
docker compose build
docker compose up
docker compose down
docker compose ps
docker compose logs
```

Testing

```
pytest -q
```

Network Verification

```
ip -4 addr show wlo1
ip route
ip neigh show 10.7.33.21
sudo iptables -S INPUT
sudo tc class show dev wlo1
```

Bandwidth Cleanup

For the controlled laboratory setup:

```
sudo tc qdisc del dev wlo1 root
```